你很快就可以使用了。立即注册，开始你的 Notion 创造之旅吧。 注册或登录

Stanford CS336 Assignment 1

> **周期：3 周。核心目标：不是调包训练模型，而是亲手把一个 Transformer Language Model 从 tokenizer 一直写到训练和生成。**

🎯 **这次作业最重要的不是跑出多好的结果，而是搞明白一个 LLM 到底是怎么工作的。**

作业完成后，你应该能够独立解释：**文本 → Token → Embedding → Transformer → Logits → Cross Entropy → Backprop → AdamW → Generation**。

0. 你到底要做什么?

本次作业基于 Stanford CS336 的 **Assignment 1: Basics**。

官方 GitHub

2026 Assignment PDF

Stanford 原版包含 TinyStories、OpenWebText、resource accounting 和 leaderboard。**我们这次不追求 leaderboard，也不要求极限性能优化。**

你需要完成三件事：

1. **自己实现 BPE Tokenizer**
2. **自己实现 Transformer Language Model**
3. **自己实现训练所需的 loss、optimizer、training loop，并真正把一个小模型训练起来**

最终要求：**代码能跑 + 测试通过 + 模型能训练 + 你本人能解释。**

---

1. 第一步：先把官方项目跑起来

1.1 下载代码

```
git clone https://github.com/stanford-cs336/assignment1-basics.git cd
assignment1-basics
```

建议使用 Linux / Ubuntu 环境。

1.2 安装环境

官方项目使用 `uv` 管理 Python 环境。

```
pip install uv uv run pytest
```

第一次运行出现大量失败或 `NotImplementedError` 是正常的，因为你的任务就是逐步把缺失部分实现出来。

1.3 下载 TinyStories

```
mkdir -p data cd data wget
https://huggingface.co/datasets/roneneldan/TinyStories/resolve/main/TinyS
GPT4-train.txt wget
https://huggingface.co/datasets/roneneldan/TinyStories/resolve/main/TinyS
GPT4-valid.txt cd ..
```

先只做 TinyStories。OpenWebText 放到最后，有余力再做。

---

2. Week 1： Tokenizer

本周目标

实现一个 **Byte-level BPE Tokenizer**。

你至少要真正搞懂：

- 原始文本为什么不能直接送进 Transformer?
- byte 是什么?
- vocabulary 是什么?
- BPE 为什么要不断 merge?
- encode 在做什么?
- decode 在做什么?
- special token 是什么?

必须完成

- [ ] BPE vocabulary training
- [ ] merge 规则
- [ ] encode
- [ ] decode
- [ ] special tokens
- [ ] 对应 tokenizer tests 通过
- [ ] 在 TinyStories 上实际训练 tokenizer

你应该能回答

- BPE 的输入和输出分别是什么?
- 为什么使用 subword tokenizer?
- vocabulary size 太大或太小分别有什么影响?
- 一段字符串经过 tokenizer 后到底变成了什么?

> ✅ **Week 1 完成标准：给你一句没见过的话，你的 tokenizer 能 encode 成 token IDs，再 decode 回基本一致的文本。**

---

3. Week 2： Transformer

这是本作业最重要的一周。

需要自己实现的核心组件

- [ ] Linear
- [ ] Embedding
- [ ] RMSNorm
- [ ] SwiGLU / Feed Forward Network
- [ ] Rotary Positional Embedding（RoPE）
- [ ] Scaled Dot-Product Attention
- [ ] Causal Multi-Head Self-Attention
- [ ] Transformer Block
- [ ] 完整 Transformer Language Model
- [ ] 对应 model tests 通过

不允许直接调完整现成模型

例如不要直接使用：

```
transformers.AutoModelForCausalLM torch.nn.Transformer
```

基础 tensor operation 和 PyTorch autograd 可以正常使用。

做完以后至少要能解释

Attention

给定输入 `X: [batch, sequence_length, hidden_dim]`，你应该知道它如何经过：

`Q, K, V → Attention Score → Causal Mask → Softmax → Weighted Sum`

并能说清楚关键 tensor shape 怎么变化。

RoPE

- 为什么 Transformer 需要位置信息?
- RoPE 作用在谁身上?
- 为什么通常作用于 Q 和 K?
- RoPE 是否有需要训练的参数?

Causal Mask

你应该知道为什么位置 t 不能看到 t+1，以及如果去掉 causal mask 会发生什么。

Transformer Block

你应该能解释：

```
RMSNorm → Attention → Residual → RMSNorm → FFN → Residual
```

✅ **Week 2 完成标准：不仅 forward 能跑，而且随机问一个 tensor 的 shape，你能够沿着模型把它推出来。**

---

4. Week 3：真正训练一个 Language Model

需要实现

- [ ] Cross Entropy Loss
- [ ] AdamW
- [ ] Learning Rate Schedule
- [ ] Data Sampling / Batch Construction
- [ ] Training Loop
- [ ] Checkpoint Save / Load
- [ ] Validation

- [ ] Text Generation

如果官方 Assignment 对 gradient clipping 有要求，也一并实现。

最终必须完成：TinyStories

使用你自己实现的 **Tokenizer + Transformer + Loss + AdamW + Training Loop** 在 TinyStories 上真正训练一个语言模型。

至少展示：

1. training loss 随训练下降；
2. validation loss；
3. 保存 checkpoint；
4. 从 checkpoint 恢复模型；
5. 给模型一个 prompt，让模型继续生成文本。

例如：

```
Once upon a time
```

模型生成效果不要求达到大模型水平。我们主要看整个训练链路是否真的工作。

---

5. OpenWebText 和性能优化：选做

以下内容作为 **Optional / Bonus**：

- OpenWebText 训练
- 更大的模型
- mixed precision
- FlashAttention
- compile
- profiling
- throughput 优化
- resource accounting

- Stanford leaderboard

如果必做内容没有完成，**不要先做这些。**

> **优先级：Correctness > Understanding > Training > Performance**

---

6. 最终需要提交什么？

每个人提交一个文件夹或 Git Repository，至少包含以下四项。

① 完整代码

README 中写清楚：

```
Environment How to run tests How to train tokenizer How to train model
How to evaluate How to generate text
```

② Test Result

运行：

```
uv run pytest
```

保存最终测试结果截图或 log。与本次必做部分相关的 correctness tests 原则上应通过。

③ Training Result

至少包含：

- Training loss curve
- Validation loss
- 模型参数量

- 训练配置
- 最终生成示例

训练配置至少写：

```
vocab_size context_length d_model num_layers num_heads batch_size
learning_rate training_steps
```

④ 一页实验总结

不要写长报告，一页左右即可。回答：

1. 你最终训练了一个多大的模型?
2. loss 是否正常下降?
3. 训练过程中遇到的最大 bug 是什么?
4. 你是怎么定位并解决这个 bug 的?
5. 做完整个 Assignment 后，你对 Transformer 哪一部分理解变化最大?

---

7. AI 使用规则

可以使用 ChatGPT、Claude、Copilot 等 AI 工具，但原则很简单：

> **AI 可以帮助你学习和 debug，但不能替你理解。**

允许：

- 解释概念
- 解释报错
- 查询 PyTorch API
- 帮助理解 tensor shape
- 讨论为什么某段代码不工作
- 帮助检查实现逻辑

不要把整个 Assignment 直接交给 AI 一次性生成，也不要复制网上完整 solution。

最终验收时，**你提交的每一段核心代码都默认视为你能够解释。**

如果代码能跑，但本人无法解释核心逻辑，则该部分视为没有真正完成。

---

8. 最终验收

每个人安排约 **15 分钟个人验收**，不用做 PPT，直接打开代码和实验结果。

① 代码

随机打开一部分代码，请你解释它在做什么。

② 原理

可能随机问：

- Attention 为什么除以 sqrt(d_k)?
- Q、K、V 分别是什么?
- causal mask 为什么需要?
- RoPE 到底对什么做了旋转?
- RMSNorm 和 LayerNorm 有什么区别?
- Cross Entropy 的 logits 是什么 shape?
- AdamW 和 Adam 有什么区别?
- language model 为什么是在预测 next token?

③ Debug

例如：

> 如果训练 loss 完全不下降，你会从哪里开始查?

不要求现场写大量代码，主要看你是否真正理解自己的实现。

---

9. 三周安排

时间	任务	完成标志
Week 1	BPE Tokenizer	Tokenizer tests + TinyStories tokenizer
Week 2	Transformer	Model tests + forward 正确
Week 3	Training	TinyStories loss 下降 + checkpoint + generation

---

10. 最后一句

这次作业不要求你成为 LLM 系统工程师。

真正希望你完成之后能够做到的是：

> **以后再看到 Transformer、RoPE、Tokenizer、AdamW、Cross Entropy、Language Modeling 时，不再只是“听说过”，而是知道它们在代码中到底怎么工作的。**

先把 TinyStories 完完整整跑通，再考虑更大的模型、更大的数据和更复杂的方法。